Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería, Ingeniería en Ciencias y Sistemas

Laboratorio Compiladores 2

Contacto 3xtrat3rr3str3D
Manual Técnico

Fase 2

Octubre 28 del 2,026

Enmer Oswaldo Sandoval Mazariegos 202031773

MANUAL TÉCNICO: COMPILADOR CONTACTO 3XTRAT3RR3STR3D (JAVA SWING + ANTLR4)

Requisitos

- JDK 17 o superior.
- Apache Maven 3.x, ejecutado con el JDK 17. El plugin de ANTLR 4.13.2 no funciona con Java 8.
- GCC (MinGW-w64), opcional, para compilar el código C generado.
- 4 GB de RAM.
- IDE compatible con Java (IntelliJ IDEA o NetBeans).

1) RESUMEN DEL PROYECTO

Aplicación de escritorio en Java Swing que compila programas escritos en tres lenguajes: Y? (.y), Zetariano (.z) y Pig Latin (.pig). El archivo .pig principal es el punto de entrada e importa los archivos de los otros dos lenguajes. Permite:

- Crear carpetas y archivos .pig, .y y .z; abrir proyectos y archivos; guardar; descargar archivos y carpetas.
- Editar el código en un JTextPane con coloreado en tiempo real para los tres lenguajes, sin librerías.
- Compilar: léxico y sintaxis (ANTLR4), AST (visitors), semántico (3 pasadas), cuartetas, código de tres direcciones en C y código RISC-V.
- Ejecutar el código RISC-V en un simulador propio, escrito en Java, sin simuladores externos.
- Mostrar la tabla de errores (tipo, archivo, línea, columna, lexema y descripción).
- Mostrar los reportes: tabla de símbolos, tabla de tipos, tabla de compatibilidad, cuartetas, código de tres direcciones y código RISC-V.

Variaciones respecto a la fase 1

- Zetariano: herencia simple con extends, @Override obligatorio, super(...), super.metodo(), this(...), reglas de acceso sin paquetes (protected solo para la clase y sus hijas, sin modificador igual que private) y polimorfismo con despacho dinámico.
- Pig Latin: declaración de objetos con tipo explícito (esto x : novus C(...) : Tipo;), literal null y tipo bool.
- Y?: el elegir ya no usa llaves y los dos puntos son opcionales; las funciones pueden retornar arreglos.
- Código C: ya no usa funciones de C. Todo el programa queda dentro de main y las llamadas se hacen con goto y una etiqueta de retorno guardada en el stack.
- Errores en tiempo de ejecución: heap agotado y uso de un objeto nulo.
- Nueva generación de código RISC-V (programa.s) y simulador RISC-V propio.
- Tabla de tipos y tabla de compatibilidad generales en un JTable simple, ya no una por lenguaje.
- Barra de estado con la línea y columna del cursor.

2) TECNOLOGÍAS UTILIZADAS

- Java (JDK) 17 o superior: lenguaje del proyecto. Se compila con release 17.
- Java Swing (incluido en el JDK): interfaz gráfica, diseñada con el editor visual de NetBeans (IDERecaf).
- ANTLR 4.13.2: genera los lexers y parsers de Y?, Zetariano y Pig Latin. El AST se arma con visitors.
- Apache Maven 3.x: construcción del proyecto. antlr4-maven-plugin genera los analizadores y maven-shade-plugin 3.5.1 arma el JAR con sus dependencias.
- Lombok 1.18.46: genera getters y setters (@Getter, @Setter).
- GCC (MinGW-w64): compila el programa.c generado.

3) CONSTRUCCIÓN DEL PROYECTO

```
mvn antlr4:antlr4
mvn clean package
```

El primer comando genera los analizadores en target/generated-sources/antlr4. El segundo compila todo y deja el ejecutable en target/Proyecto_Fase1.jar.

En IntelliJ IDEA el SDK del proyecto, el JRE del runner de Maven y el JDK del importador de Maven deben ser 17 o superior. Después de generar los analizadores se recarga el proyecto Maven para que reconozca las clases generadas.

4) ARQUITECTURA

4.1 Paquetes principales

practica.compi2

- Main: punto de entrada. Sin argumentos abre el IDE; con un archivo compila por consola.

practica.compi2.vista

- IDERecaf: JFrame principal con el árbol de proyecto, editor, consola, menús, botón Compilar y barra de estado.
- IdeConsole: escribe los mensajes en la consola del IDE.

practica.compi2.vista.controllers

- ProjectController: abrir, crear y cerrar proyectos; nuevo archivo; árbol de trabajo.
- EditorController: abrir, guardar y guardar como.
- CompileController: compila, muestra errores y reportes, descarga el código C y RISC-V, y ejecuta el simulador en un SwingWorker para no bloquear la interfaz.
- ConsoleSimulatorIO: conecta el simulador con la consola del IDE. La salida se escribe en la consola y la entrada se pide con un diálogo.
- FolderController: nueva carpeta, descargar archivo y descargar carpeta.
- FileTreeBuilder, FileNode: construyen el árbol de archivos (JTree).

practica.compi2.vista.editor

- SyntaxHighlighter: coloreado en tiempo real sin librerías, recorre los tokens del lexer de ANTLR.
- TokenStyles, LexerFactory: estilo de cada tipo de token y lexer según la extensión.
- CaretStatus: muestra la línea, la columna y el total de líneas en la barra de estado.
- FileTemplates: plantilla inicial de cada archivo nuevo.

practica.compi2.vista.reports

- ErrorsDialog: tabla de errores.
- ResultsDialog: pestañas de tabla de símbolos, tabla de tipos, compatibilidad, cuartetas, código de tres direcciones y RISC-V.
- TableFactory: crea las JTable de los reportes, no editables y con la cuadrícula visible.

practica.compi2.compiler / parsers

- Compiler, CompilationResult: coordinan las fases y guardan sus resultados.
- ProgramLoader: carga el .pig principal y resuelve los import de forma recursiva. También carga la clase padre de cada clase que usa extends.
- PigLatinFileParser, YFileParser, ZetarianoFileParser: ejecutan el lexer y parser de ANTLR de cada archivo.

practica.compi2.antlr4 (generados por ANTLR)

- YLexer / YParser, ZetarianoLexer / ZetarianoParser, PigLatinLexer / PigLatinParser: analizadores léxicos y sintácticos.

- YLexerBase: maneja la indentación de Y? (INDENT / DEDENT).

practica.compi2.visitors

- YProgramVisitor, ZetarianoClassVisitor, PigLatinProgramVisitor: convierten el árbol de ANTLR en el AST.
- StatementVisitor, ExpressionVisitor, ControlFlowVisitor, AtomVisitor y TypeMapper de cada lenguaje: un visitor por parte del lenguaje.

practica.compi2.ast

- Ast, AstNode, Position: raíz del AST, nodo base y posición (línea, columna, lexema).
- programs, definitions: programas, clases, estructuras, funciones, métodos y constructores.
- statements, expressions, operations: sentencias, expresiones y operaciones. Cada nodo tiene validate, toIntermediateCode y print.
- SuperAccess: expresión super, el mismo objeto que this visto como la clase padre.
- ConstructorCall: sentencias super(...) y this(...) dentro de un constructor.

practica.compi2.semantic

- SemanticAnalyzer: hace las 3 pasadas del análisis semántico y ordena las clases para procesar primero a los padres.
- SemanticContext, TypeResolver, CallMatcher, AssignmentChecker: ámbitos, tipos, sobrecarga, asignaciones y reglas de acceso.
- symbols: Scope, Frame, GlobalTable, Symbol, ClassSymbol, FunctionSymbol. Tabla de símbolos, posiciones en el stack, jerarquía de clases e id de cada clase.
- types: TypeTable, TypeRules y las tablas de compatibilidad de cada lenguaje.

practica.compi2.c3d

- IntermediateCodeGenerator, IntermediateCodeUtils, IdsCounter: generan las cuartetas, temporales y etiquetas.
- TemporalSaver, CallSite: guardan en el stack los temporales vivos alrededor de cada llamada.
- ReturnLinker: agrega al final de cada función los saltos de regreso a cada lugar desde donde fue llamada.
- cuartetas: Operation3D, Assignation3D, Conditional3D, Goto3D, Call3D, FunctionJump3D, Print3D, Halt3D, etc. Cada una sabe escribirse en C (toCCode), en RISC-V (toRiscV) y como fila de la tabla (toRow).
- access: accesos a stack, heap, temporales, punteros y literales (FrameOffset3D es el desplazamiento del marco de una llamada).
- helpers: CallHelper (llamadas), DispatchHelper (despacho dinámico), MemoryHelper (reservas en el heap y límite), NullCheckHelper (objetos nulos), CopyHelper (copia de estructuras y arreglos), entre otros.
- natives: las funciones nativas escritas en cuartetas.
- CCodeGenerator: convierte las cuartetas en programa.c y llama a gcc.

practica.compi2.riscv

- RiscVGenerator: arma programa.s con la sección .data y la sección .text.

- RiscVBuilder: escribe instrucciones y etiquetas, guarda las cadenas y decimales constantes y carga valores en registros.

practica.compi2.riscv.simulador

- Assembler: lee el texto de programa.s en dos pasadas y lo convierte en instrucciones.
- Op: las instrucciones soportadas y el tipo de cada operando.
- Instruction, Registers, DataSection, AssembledProgram: instrucción ya leída, nombres de registros, sección .data y programa listo para ejecutar.
- Simulator: ejecuta las instrucciones con 32 registros enteros, 32 registros decimales y la memoria de datos.
- RiscVRunner, SimulatorIO, StreamIO, RiscVException: ejecución, entrada y salida, y errores del simulador.

practica.compi2.errors / console

- ErrorManager, CompilationError, SyntaxErrorListener, AntlrMessageTranslator: registro de errores y traducción de los mensajes de ANTLR.
- ConsoleRunner, ConsoleOptions y los printers: modo consola con AST, tablas, cuartetas, código C, RISC-V y ejecución en el simulador.

5) FLUJO DE COMPILACIÓN (COMPILER)

1. **Carga de archivos (ProgramLoader).** Parte del .pig y sigue cada import de forma recursiva. La ruta modelos.Perro.z se busca en la carpeta modelos/ del proyecto. Si una clase hereda de otra que no se importó, la clase padre se busca primero en la misma carpeta y luego en todo el proyecto. Cada archivo se analiza con su lexer y parser de ANTLR y un SyntaxErrorListener, y los visitors arman el AST. Si hay errores léxicos o sintácticos se muestra la tabla de errores y termina.
2. **Semántico (SemanticAnalyzer).** registerTypes registra estructuras y clases en la TypeTable. Después se enlaza cada clase con su padre, se detecta herencia cíclica o un padre que no existe, y las clases se ordenan por profundidad para procesar primero a los padres. registerMembers registra campos, atributos, funciones, métodos y constructores, y revisa las reglas de @Override. validatePrograms ejecuta validate de cada nodo: ámbitos, tipos, sobrecarga, reglas de acceso y posición de cada variable en el stack. Si hay errores se muestra la tabla de errores y termina.
3. **Cuartetas (IntermediateCodeGenerator).** Cada nodo ejecuta toIntermediateCode y agrega sus cuartetas. Se agregan las funciones nativas, los despachadores de los métodos sobrescritos y, al final, los saltos de retorno de cada función (ReturnLinker).
4. **Código C (CCodeGenerator).** Cada cuarteta se escribe en C. El IDE guarda programa.c en la carpeta del proyecto y lo compila con gcc.
5. **Código RISC-V (RiscVGenerator).** Cada cuarteta se escribe en RISC-V. El IDE guarda programa.s en la carpeta del proyecto.
6. **Ejecución (opcional).** Con Ejecutar RISC-V, el simulador ensambla programa.s y lo ejecuta mostrando la salida en la consola.

6) MODELO DE MEMORIA DEL CÓDIGO GENERADO

- Stack de 100,000 celdas y heap de 1,000,000 celdas, todas de tipo double. P apunta al inicio del marco actual en el stack y H a la siguiente celda libre del heap. heap[0] se reserva como null y cadena vacía, por eso H inicia en 1.
- Marco de una función: la celda 0 guarda el valor de retorno, la 1 guarda this en los métodos y constructores, y después van los parámetros y las variables locales.
- Cadenas: caracteres en el heap terminados en 0. Estructuras: un bloque de celdas en el heap, una por campo. Arreglos: un bloque en el heap con las dimensiones al inicio y los datos aplanados por filas.
- Objetos: un bloque en el heap. La celda 0 guarda el id de la clase, después van los atributos heredados del padre y al final los atributos propios. Así un atributo heredado tiene la misma posición en la clase padre y en la hija.
- Las estructuras se copian por valor al declararlas o asignarlas (CopyHelper).

6.1 Llamadas sin funciones de C

Todo el programa queda dentro de main. El programa principal va primero y cada función queda como un bloque con su etiqueta. Una llamada arma el marco nuevo así:

```
[marco actual][temporales guardados][dirección de retorno][marco nuevo]
```

1. Se copian this y los argumentos al marco nuevo.
2. Se guarda en la celda anterior al marco nuevo el número de la etiqueta de retorno.
3. Se hace P = P + desplazamiento y goto a la etiqueta de la función.
4. Al terminar, la función lee la dirección de retorno y salta con if t == etiqueta goto etiqueta, una comparación por cada lugar desde donde se llamó.
5. En la etiqueta de retorno se lee stack[P], se regresa P y se restauran los temporales.

Los temporales son globales, por eso los que se usan antes y después de una llamada se guardan en el stack entre el marco actual y el nuevo (TemporalSaver). Esto permite la recursividad.

7) HERENCIA Y POLIMORFISMO EN ZETARIANO

- Herencia simple con extends. Se reporta error si la clase padre no existe o si la herencia es cíclica.
- @Override es obligatorio para sobrescribir un método del padre y es error usarlo en un método que no sobrescribe nada. El método puede retornar el mismo tipo o una subclase (retorno covariante) y no puede ser menos visible que el del padre.
- super(...) y this(...) solo pueden ir como primera instrucción de un constructor. Si el constructor del hijo no llama a super(...), se llama solo al constructor vacío del padre; si el padre no lo tiene, es error.
- super.metodo() llama directo a la versión del padre, sin despacho dinámico.
- Acceso: public desde cualquier lugar; protected en la clase y sus hijas; private y sin modificador solo dentro de la propia clase.

7.1 Despacho dinámico

Si algún método heredable tiene subclases que lo sobrescriben, la llamada pasa por un despachador llamado Clase_virtual_metodo (DispatchHelper). El despachador no crea un marco propio: lee this en P + 1, lee el id de la clase en heap[this] y lo compara con el id de cada subclase que sobrescribe el método. Cuando coincide salta a esa implementación; si ninguno coincide salta a la implementación de la clase base. Las implementaciones regresan a donde se llamó al despachador.

8) CAMBIOS EN PIG LATIN Y Y?

- Pig Latin: esto a : novus Perro("Firulais") : Animal; declara una variable de tipo Animal con un objeto Perro. También se agregó el literal null y el tipo bool.
- Y?: elegir (expresión) con dos puntos opcionales y sin llaves; los casos se definen por indentación. El tipo de retorno de una función acepta [] para retornar arreglos.

9) ERRORES EN TIEMPO DE EJECUCIÓN

- Heap agotado: cada reserva en el heap compara H con el tamaño del heap. Si se pasa, imprime "Error: heap agotado" y termina el programa (Halt3D).
- Objeto nulo: antes de usar un objeto que no es this ni super se revisa que no sea 0. Si lo es, imprime "Error en tiempo de ejecución: se usó un objeto nulo en la línea L, columna C" y termina el programa.

10) TRADUCCIÓN A RISC-V

Cada cuarteta tiene su método toRiscV. Todos los valores se manejan como double, igual que en el código C.

- Sección .data: stack, heap y temporales como bloques de bytes (.space), las cadenas constantes (.string) y los decimales constantes (.double).
- Registros base: s0 apunta al stack, s1 al heap y s2 al arreglo de temporales.
- Los punteros P y H se guardan en fs0 y fs1. ft0 a ft3 se usan para operar y t0 a t3 para direcciones y conversiones.
- Las operaciones usan fadd.d, fsub.d, fmul.d y fdiv.d. El módulo convierte a entero, usa rem y regresa a decimal.
- Las condiciones usan feq.d, flt.d o fle.d y luego beqz o bnez.
- La impresión y la lectura usan ecall: 1 imprime un entero, 4 una cadena, 11 un carácter, 12 lee un carácter y 10 termina el programa. Los decimales se imprimen con dos decimales redondeando a centavos.

11) SIMULADOR RISC-V

- Assembler, primera pasada: lee cada línea, registra las etiquetas, procesa las directivas de .data (DataSection) y convierte cada instrucción. Cada Op declara qué operandos espera: x registro entero, f registro decimal, n número, l etiqueta, m memoria offset(registro) y r modo de redondeo.
- Assembler, segunda pasada: resuelve las etiquetas de los saltos y las direcciones de la.
- Simulator: ejecuta desde la etiqueta main con 32 registros enteros (x0 siempre vale 0), 32 registros decimales y la memoria de datos. La sección .data inicia en la dirección 0x10000000.
- Detecta división entre cero, acceso a direcciones inválidas, programas que terminan sin llamar a la salida y un límite de 500,000,000 instrucciones para cortar ciclos infinitos.
- Soporta las 27 instrucciones que emite el generador: add, addi, slli, div, rem, li, la, mv, beqz, bnez, bge, j, ecall, fld, fsd, fadd.d, fsub.d, fmul.d, fdiv.d, fmsub.d, feq.d, flt.d, fle.d, fcvt.d.l, fcvt.l.d, fmv.d y fneg.d.

12) REPORTES

- Tabla de símbolos: nombre, tipo, categoría, ámbito, posición y archivo.
- Tabla de tipos: tipo, categoría, tamaño, cómo se guarda y miembros con su posición.
- Compatibilidad: una sola tabla general con operador, tipo izquierdo, tipo derecho, resultado y lenguaje. Las reglas que solo aplican en un lenguaje indican cuál.
- Cuartetas, código de tres direcciones y RISC-V.

13) MODO CONSOLA

```
java -jar Proyecto_Fase1.jar main.pig [--ast] [--tipos] [--cuartetas] [--c3d]
[--riscv] [--ejecutar]
```

Siempre imprime la tabla de símbolos y los errores. Si no hay errores guarda programa.c y programa.s en la carpeta actual. Con --ejecutar corre el programa en el simulador usando la entrada y salida de la consola.

14) ESPECIFICACIÓN DE LOS LENGUAJES

Se muestran las gramáticas ANTLR4 actuales de los tres lenguajes, sin las etiquetas de los visitors. Los literales son los mismos de la fase 1: entero, decimal, carácter, cadena, identificador y escapes.

Palabras reservadas nuevas en la fase 2

- Zetariano: extends, @Override, super.
- Pig Latin: null, bool.

Y? (.y)

```
program
    : structSection? functionSection EOF
    ;

structSection
    : SECCION_ESTRUCTURAS NEWLINE structDefinition*
    ;

functionSection
    : SECCION_FUNCIONES NEWLINE functionDefinition*
    ;

structDefinition
    : ESTRUCTURA ID COLON NEWLINE INDENT fieldDefinition+ DEDENT
    ;

fieldDefinition
    : type ID dimensions? SEMICOLON? NEWLINE
    ;

functionDefinition
    : DEFINIR ID LPAREN parameters? RPAREN (ARROW (LBRACE RBRACE | (LBRACKET
RBRACKET)+)? type)? COLON block
    ;

parameters
    : parameter (COMMA parameter)*
    ;

parameter
    : type ID
    | (LBRACKET RBRACKET)+ type ID
    | LBRACE RBRACE ID ID
    ;

block
    : NEWLINE INDENT statement+ DEDENT
```

```
    ;

statement
    : structDefinition
    | simpleStatement SEMICOLON? NEWLINE
    | controlStatement
    ;

simpleStatement
    : declaration
    | assignment
    | IMPRIMIR LPAREN expression RPAREN
    | RETORNAR expression?
    | ROMPER
    | CONTINUAR
    | expression
    ;

declaration
    : type ID dimensions? (ASSIGN expression)?
    ;

assignment
    : expression ASSIGN expression
    ;

dimensions
    : (LBRACKET expression RBRACKET)+
    ;

controlStatement
    : ifStatement
    | switchStatement
    | forStatement
    | whileStatement
    | doWhileStatement
    ;

ifStatement
    : SI LPAREN expression RPAREN ENTONCES block elseIfClause* elseClause?
    ;

elseIfClause
    : SINO LPAREN expression RPAREN ENTONCES block
    ;

elseClause
    : CONTRARIO block
    ;

switchStatement
```

```
    : ELEGIR LPAREN expression RPAREN COLON? NEWLINE INDENT caseClause*
defaultClause? DEDENT
    ;

caseClause
    : CASO expression COLON block
    ;

defaultClause
    : SIEMPRE COLON block
    ;

forStatement
    : PARA LPAREN forInit? SEMICOLON expression? SEMICOLON forUpdate? RPAREN
COLON block
    ;

forInit
    : declaration
    | assignment
    ;

forUpdate
    : assignment
    | expression
    ;

whileStatement
    : MIENTRAS LPAREN expression RPAREN HACER block
    ;

doWhileStatement
    : HACER COLON block MIENTRAS LPAREN expression RPAREN SEMICOLON? NEWLINE
    ;

expression
    : expression DOT ID
    | expression LBRACKET expression RBRACKET
    | expression op=(INC | DEC)
    | op=(NOT | MINUS) expression
    | expression op=(MULT | DIV) expression
    | expression op=(PLUS | MINUS) expression
    | expression op=(LT | GT | LE | GE) expression
    | expression op=(EQ | NE) expression
    | expression AND expression
    | expression OR expression
    | atom
    ;

atom
    : LPAREN expression RPAREN
```

```
    | literal
    | ID LPAREN arguments? RPAREN
    | ID
    | LEER LPAREN RPAREN
    | LBRACE (expression (COMMA expression)*)? RBRACE
    ;

arguments
    : expression (COMMA expression)*
    ;

literal
    : INT_LIT
    | DECIMAL_LIT
    | STRING_LIT
    | CHAR_LIT
    | VERDADERO
    | FALSO
    ;

type
    : ENTERO
    | FLOTANTE
    | CADENA
    | CARACTER
    | BOOL
    | ID
    ;
```

Zetariano (.z)

```
compilationUnit
    : classDeclaration EOF
    ;

classDeclaration
    : accessModifier? CLASS ID (EXTENDS ID)? LBRACE classMember* RBRACE
    ;

classMember
    : fieldDeclaration
    | constructorDeclaration
    | methodDeclaration
    ;

accessModifier
    : PUBLIC
    | PRIVATE
    | PROTECTED
    ;

fieldDeclaration
    : accessModifier? type ID (ASSIGN expression)? SEMICOLON
    ;

constructorDeclaration
    : accessModifier? ID LPAREN parameters? RPAREN block
    ;

methodDeclaration
    : OVERRIDE? accessModifier? (type | VOID) ID LPAREN parameters? RPAREN
block
    ;

parameters
    : parameter (COMMA parameter)*
    ;

parameter
    : type ID
    ;

type
    : (primitiveType | ID) (LBRACKET RBRACKET)*
    ;

primitiveType
    : INT
```

```
    | DOUBLE
    | CHAR
    | BOOLEAN
    | STRING
    ;

block
    : LBRACE statement* RBRACE
    ;

statement
    : block
    | simpleStatement SEMICOLON
    | controlStatement
    | SEMICOLON
    ;

simpleStatement
    : localDeclaration
    | assignment
    | (PRINTLN | PRINT) LPAREN expression? RPAREN
    | SUPER LPAREN arguments? RPAREN
    | THIS LPAREN arguments? RPAREN
    | RETURN expression?
    | BREAK
    | CONTINUE
    | expression
    ;

localDeclaration
    : type ID (ASSIGN expression)?
    ;

assignment
    : expression op=(ASSIGN | ADD_ASSIGN | SUB_ASSIGN | MUL_ASSIGN |
DIV_ASSIGN) expression
    ;

controlStatement
    : IF LPAREN expression RPAREN statement (ELSE statement)?
    | SWITCH LPAREN expression RPAREN LBRACE switchSection* RBRACE
    | FOR LPAREN forInit? SEMICOLON expression? SEMICOLON forUpdate? RPAREN
statement
    | WHILE LPAREN expression RPAREN statement
    | DO statement WHILE LPAREN expression RPAREN SEMICOLON
    ;

switchSection
    : (CASE expression | DEFAULT) COLON statement*
    ;
```

```
forInit
    : localDeclaration
    | assignment
    ;

forUpdate
    : assignment
    | expression
    ;

expression
    : expression DOT ID LPAREN arguments? RPAREN
    | expression DOT ID
    | expression LBRACKET expression RBRACKET
    | expression op=(INC | DEC)
    | op=(INC | DEC) expression
    | op=(NOT | MINUS) expression
    | expression op=(MULT | DIV | MOD) expression
    | expression op=(PLUS | MINUS) expression
    | expression op=(LT | GT | LE | GE) expression
    | expression op=(EQ | NE) expression
    | expression AND expression
    | expression OR expression
    | <assoc=right> expression QUESTION expression COLON expression
    | atom
    ;

atom
    : LPAREN expression RPAREN
    | literal
    | THIS
    | SUPER
    | NULL
    | ID LPAREN arguments? RPAREN
    | ID
    | NEW ID LPAREN arguments? RPAREN
    | NEW (primitiveType | ID) (LBRACKET expression RBRACKET)+
    | LBRACE (expression (COMMA expression)*)? RBRACE
    | READLN LPAREN RPAREN
    ;

arguments
    : expression (COMMA expression)*
    ;

literal
    : INT_LIT
    | DECIMAL_LIT
    | STRING_LIT
    | CHAR_LIT
    | TRUE
```

```
| FALSE
;
```

Pig Latin (.pig)

```
program
    : importStatement* variablesSection? mainSection EOF
    ;

importStatement
    : IMPORT ID (DOT ID)+ SEMICOLON?
    ;

variablesSection
    : VARIABILES declaration*
    ;

mainSection
    : MAIOR statement* FINIS_MAIOR SEMICOLON
    ;

declaration
    : ESTO ID COLON expression COLON type SEMICOLON
    | ESTO ID COLON type? expression? SEMICOLON
    | SERIES ID dimensions COLON type expression? SEMICOLON
    ;

dimensions
    : (LBRACKET expression RBRACKET)+
    ;

block
    : LBRACE statement* RBRACE
    ;

statement
    : declaration
    | simpleStatement
    | controlStatement
    ;

simpleStatement
    : assignment SEMICOLON
    | PRINT expression (PRINT expression)* SEMICOLON?
    | expression? READ SEMICOLON?
    | PERGE SEMICOLON
    | INTERRUMPE SEMICOLON
    | expression SEMICOLON
    ;

assignment
    : expression ASSIGN expression
```

```
    ;

controlStatement
    : SI LPAREN expression RPAREN block elseIfClause* elseClause? FINIS
SEMICOLON
    | DUM LPAREN expression RPAREN block FINIS SEMICOLON
    | FACERE block DUM LPAREN expression RPAREN SEMICOLON
    | PER LPAREN forInit? SEMICOLON expression? SEMICOLON forUpdate? RPAREN
block (FINIS SEMICOLON)?
    ;

elseIfClause
    : ALITER LPAREN expression RPAREN block
    ;

elseClause
    : ALITER block
    ;

forInit
    : ESTO ID COLON type? expression
    | assignment
    ;

forUpdate
    : assignment
    | expression
    ;

expression
    : expression DOT ID LPAREN arguments? RPAREN
    | expression DOT ID
    | expression LBRACKET expression RBRACKET
    | expression op=(INC | DEC)
    | op=(NOT | MINUS) expression
    | expression op=(MULT | DIV) expression
    | expression op=(PLUS | MINUS) expression
    | expression op=(LT | GT | LE | GE) expression
    | expression op=(EQ | NE) expression
    | expression AND expression
    | expression OR expression
    | atom
    ;

atom
    : LPAREN expression RPAREN
    | literal
    | ID LPAREN arguments? RPAREN
    | ID
    | NOVUS ID LPAREN arguments? RPAREN
    | NULL
```

```
    | LBRACE (expression (COMMA expression)*)? RBRACE
    ;

arguments
    : expression (COMMA expression)*
    ;

literal
    : INT_LIT
    | DECIMAL_LIT
    | STRING_LIT
    | CHAR_LIT
    | VERUM
    | FALSUS
    ;

type
    : NUMERUS
    | DECIMALIS
    | TEXTUM
    | LITTERA
    | BOOL
    | ID
    ;
```

GRACIAS